

Software Architecture
(T630019402)

Lecture #10, April 28, 2026

Jukka Ruohonen

On the Exam, Again

- ▶ As already announced on *itslearning*, the exam will allow the use of smartphones for digitalizing drawings
- ▶ Otherwise, both the exam and re-exam have 16 questions
- ▶ There are 1, 2, 3, 4, and 5 point questions
- ▶ The exam and re-exam both have a maximum of 34 points, and, therefore, to pass the course with a grade 2, you will need 17 points at minimum
- ▶ Anything that has appeared in the lectures may appear in the exam – *if you have not attended and only read the slides afterwards, you must also understand what is in the slides and further show ability to apply what is in the slides*

Today's Lecture

- ▶ Today's lecture is about **usability** and **power consumption** as distinct quality attributes
- ▶ Usability is important to cover because it is quite distinct from other QAs, being fundamentally a non-technical QA
- ▶ Power consumption is increasingly important not least because of **sustainability** and the **climate change**, but in this course it is covered as a QA because it allows to recapitulate some earlier QAs and illustrate the trade-offs involved

Question

How would you characterize usability?

Framing Usability (1/6)

- ▶ *“Usability is concerned with how easy it is for the user to accomplish a desired **task** and the kind of user **support** that the system provides”* (Bass et al. 2021, boldfacing added)
- ▶ The above definition is rather traditional in a sense that it emphasizes tasks; hence, alternatively, also concepts such as **enjoyment**, **satisfaction**, and **joy** could be emphasized
- ▶ Yet, note that all these have antonyms too; computers and IT in general are known to also cause frequent **frustration** and even **anger** among users

Framing Usability (2/6)

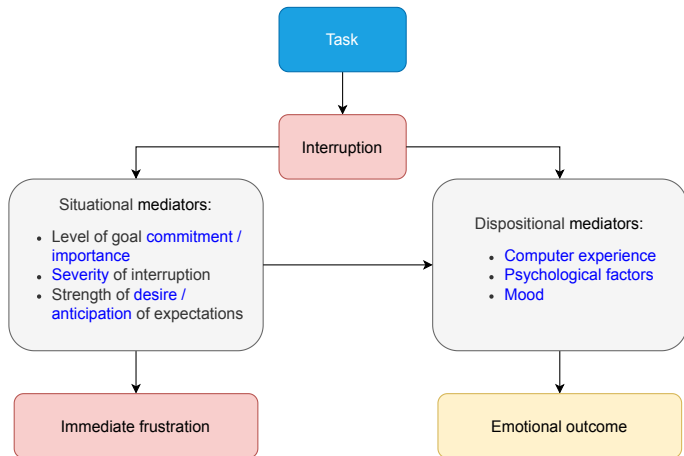


Figure 1: Computer Frustration Model (Hertzum & Hornæk 2023, Fig. 1)

Framing Usability (3/6)

- ▶ Regarding the “traditional”, task-oriented framing, the **technology acceptance model** (TAM) pioneered by Davis (1989) has been widely adopted for modeling adoption willingness among users
- ▶ Although the TAM model has been also criticized and extended, it could be argued that it is among the few theories in **information systems** research that stood the test of time in terms of its general theoretical building blocks
 - Hence, note also that information systems $\neq$ software engineering as a scientific discipline

Framing Usability (4/6)

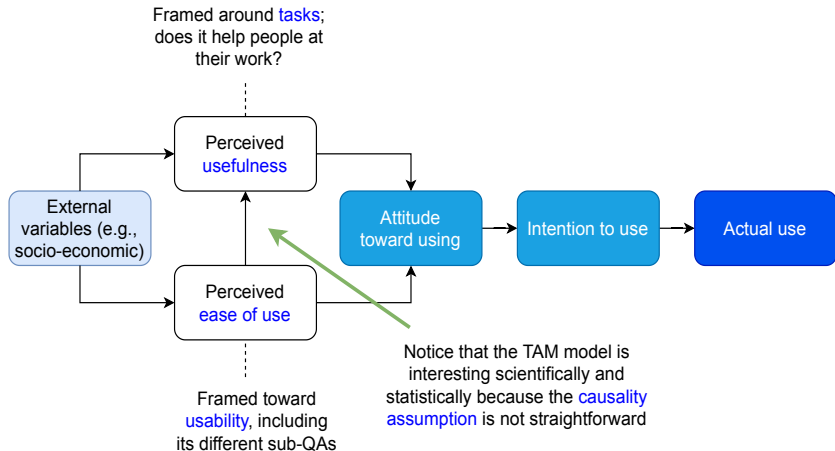


Figure 2: The TAM Model (Davis 1989)

Framing Usability (5/6)

<u>DESIGNER</u>	<u>WHAT THEY ARE RESPONSIBLE FOR</u>
UI	ELEMENTS OF THE INTERFACE THAT THE USER ENCOUNTERS
UX	THE USER'S EXPERIENCE OF USING THE INTERFACE TO ACHIEVE GOALS
UZ	THE PSYCHOLOGICAL ROOTS OF THE USER'S MOTIVATION FOR SEEKING OUT THE INTERACTION
U α	THE USER'S SELF-ACTUALIZATION
U Ω	THE ARC OF THE USER'S LIFE
U ∞	LIFE'S EXPERIENCE OF TIME
U●	THE ARC OF THE MORAL UNIVERSE

Figure 3: <https://xkcd.com/2141/>

Framing Usability (6/6)

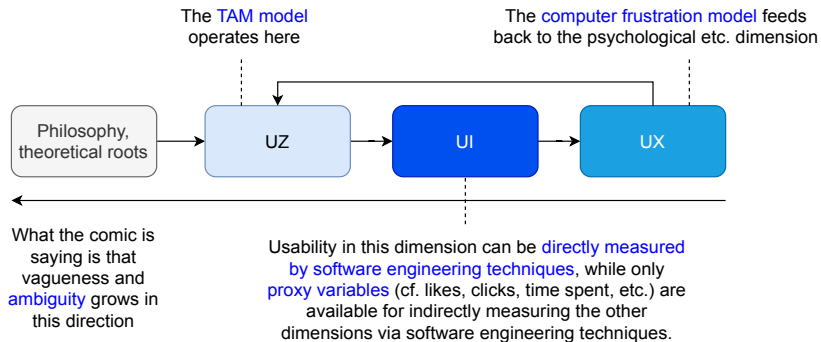


Figure 4: Thinking About the Concepts Analytically

Usability Breakdown

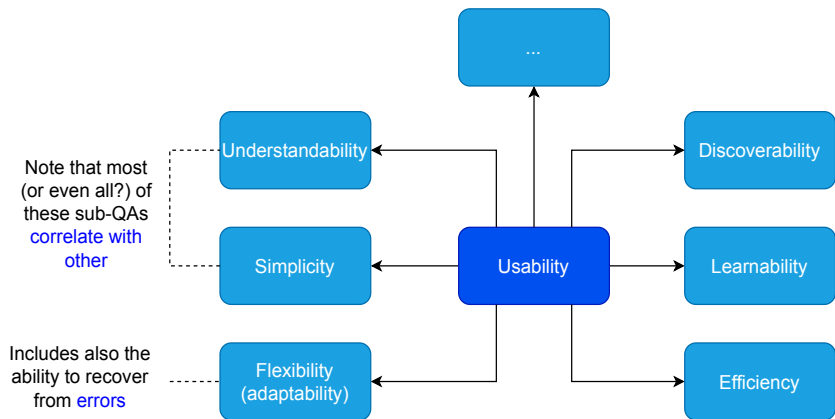


Figure 5: Some Sub-QAs for the Main Usability QA

Rethinking the TAM Model (1/2)

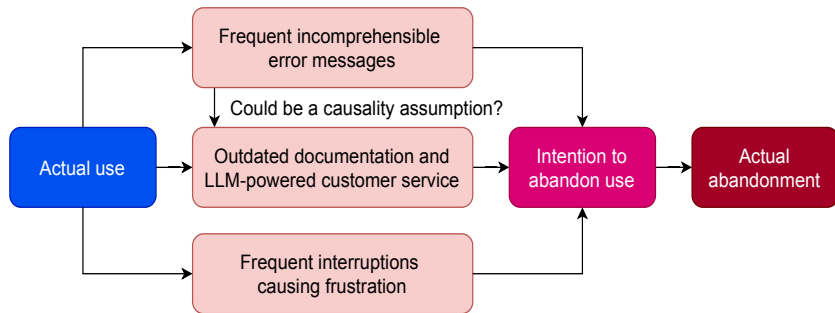


Figure 6: Thus, Recall Customer Retention Too!

Rethinking the TAM Model (2/2)

- ▶ Thus, it can be argued that one should seek to **maximize the TAM's usability and usefulness** dimensions and **minimize negative dimensions** involved
- ▶ Regarding the TAM's usefulness (task) dimension, Bass et al. (2021) recommend to provide **cancel, undo, pause/resume,** and **automation** functionalities
- ▶ Regarding the TAM's ease of use (usability) dimension, **personalization** would be a good example for a tactic

Usability Patterns: Memento (1/2)

- ▶ The so-called **memento** pattern can be used to implement different **undo** functionalities
- ▶ While simple – like all design patterns are (or should be), it is useful to note because it correlates with the **rollback** tactic from the fourth lecture
- ▶ Analytically, it can be seen to somewhat resemble also the **claim-check** pattern from the sixth lecture

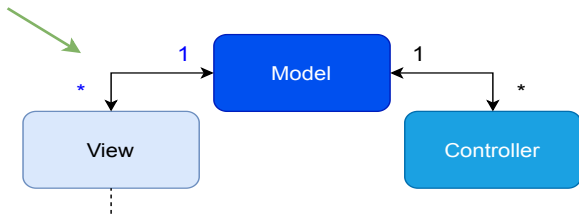
Usability Patterns: Memento (2/2)

```
# A partial serialization-based memento with R.  
#  
fn <- "a_part_of_a_heavy_computation.RData";  
  
if (exists(as.character(substitute(part.a))) == 0) {  
  
  if (file.exists(fn) == TRUE) {  
    part.a <- load(fn);          # This takes a second.  
  } else {  
    part.a <- compute.a();      # This took two days.  
    save(part.a, file = fn);  
  }  
}
```

Usability Patterns: MVC

- A recap from the third lecture

The notation is
there for a reason



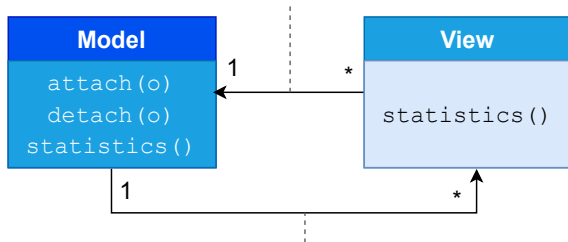
WordPress templates, skins, and
whatnot, which can be rolled without
touching the model and the controller

Figure 7: An Illustration of the MVC Pattern, Again

Usability Patterns: Observer (1/2)

- ▶ Another recap from the third lecture

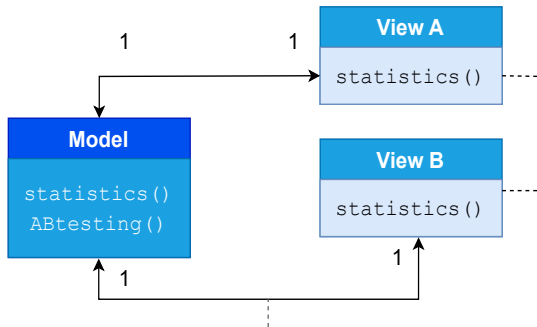
A view (a.k.a. observer) instance calls `attach(this)` and `detach(this)` to attach or detach itself itself, respectively



For each `o` that has called `attach(o)`,
call `o.statistics()` to gather statistics

Figure 8: What Might a Rationale for This Design Be?

Usability Patterns: Observer (2/2)



So, what really is a "view"? Say, in this example there might be some "main view" (such as a WordPress template), whereas the View A and View B could be some fine-grained alterations to it.

For each `o` that has called `attach(o)`,
call `o.statistics()` to gather
statistics in order to perform A/B testing

Figure 9: Yet Another Recap

Yet, Usability for Whom? (1/4)

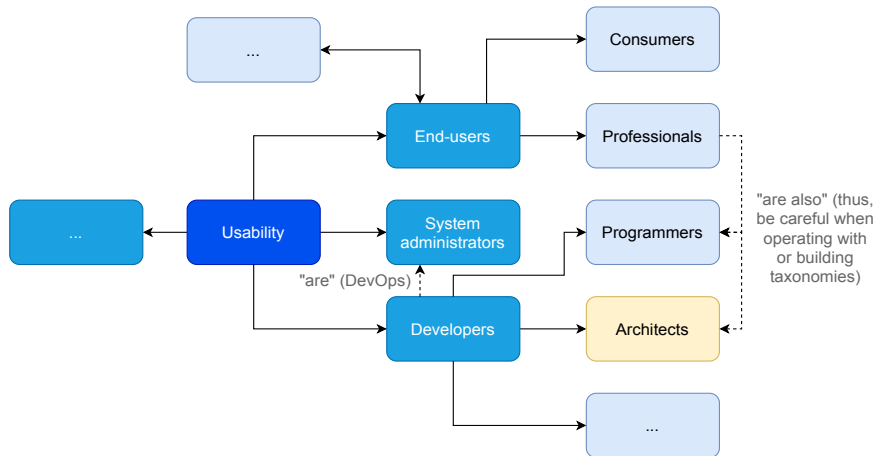


Figure 10: Like With Writing: Know Your Audience

Yet, Usability for Whom? (2/4)

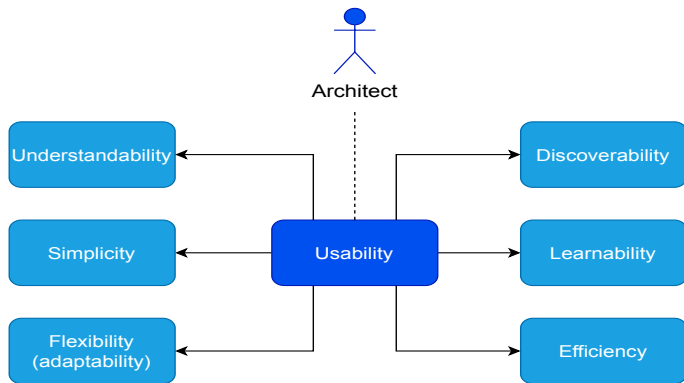


Figure 11: (a) Which Might Matter and (b) Are There Synonyms?

Yet, Usability for Whom? (3/4)

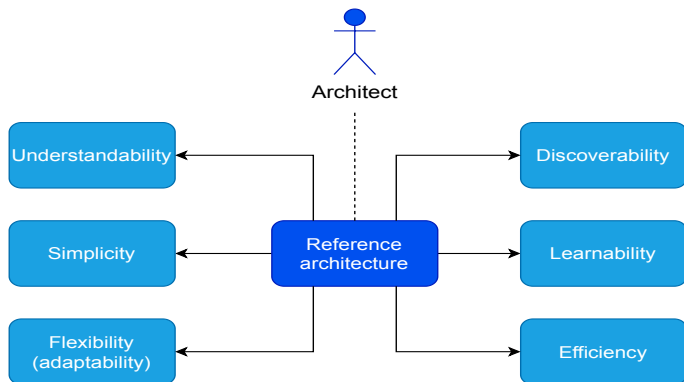


Figure 12: We Could Use These Concepts to Evaluate Reference Architectures and Their Documentation, Among Other Things

Yet, Usability for Whom? (4/4)

- ▶ Following Silva et al. (2023), we might evaluate reference architectures – or our own architectures adopted for new settings – via different **suitability** metrics
- ▶ A simple example (ibid.) would be $y = A/B$, where A is the number of components in a reference architecture that fitted well into a given domain and B the total number of components in the actual, implemented architecture

On Usability Metrics

- ▶ Usability is difficult to evaluate; often, something like focus groups, laboratory experiments, observatory studies, or related study setups are required for robust evidence
- ▶ Yet, you can automate some evaluations

pypl v0.7.13 main passing downloads rate limited by upstream service

Textstat is an easy to use library to calculate statistics from text. It helps determine readability, complexity, and grade level.



Figure 13: <https://pypi.org/project/textstat/>

Yes, Some Scientists and Engineers Pride Their Writing



Red Hat Technical Writing Style Guide

Style Conventions for Writers and Editors

Edition 7.2

Legal Notice

Copyright © 2026 Red Hat, Inc.

Figure 14: <https://stylepedia.net/style/>

Personas (1/5)

- ▶ So-called **personas** are a popular way to model usability (and other) requirements especially when you do not have means or resources to do customer surveys or when you do not even know your customer segment
- ▶ In essence: like in role playing games, try create typical “characters” (personas) mimicking your future users
- ▶ Might be a good target for a LLM?

Personas (2/5)

If the task would be to design a recommender system for scientific papers, what would these two personas tell you?

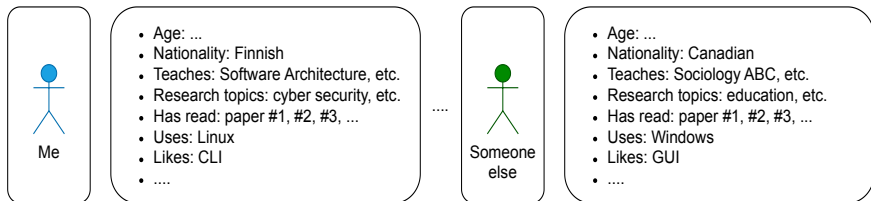
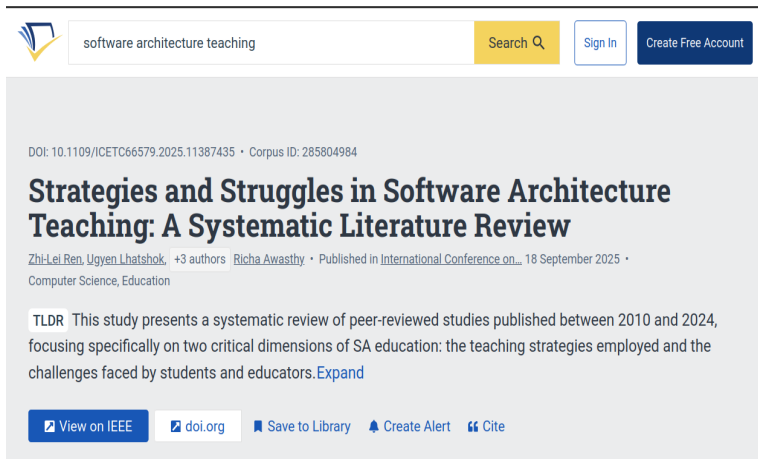


Figure 15: Two Personas and a Question

Personas (3/5)



The screenshot shows the Semantic Scholar interface. At the top, there is a search bar with the text "software architecture teaching" and a yellow "Search" button. To the right of the search bar are buttons for "Sign In" and "Create Free Account". Below the search bar, the DOI "10.1109/ICETC66579.2025.11387435" and Corpus ID "285804984" are displayed. The main title of the paper is "Strategies and Struggles in Software Architecture Teaching: A Systematic Literature Review". Below the title, the authors "Zhi-Lei Ren, Ugyen Lhatshok, +3 authors Richa Awasthy" are listed, followed by the publication information "Published in International Conference on... 18 September 2025" and the field "Computer Science, Education". A TLDR box contains the text: "This study presents a systematic review of peer-reviewed studies published between 2010 and 2024, focusing specifically on two critical dimensions of SA education: the teaching strategies employed and the challenges faced by students and educators." followed by an "Expand" link. At the bottom, there are buttons for "View on IEEE", "doi.org", "Save to Library", "Create Alert", and "Cite".

software architecture teaching

Search

Sign In

Create Free Account

DOI: 10.1109/ICETC66579.2025.11387435 • Corpus ID: 285804984

Strategies and Struggles in Software Architecture Teaching: A Systematic Literature Review

Zhi-Lei Ren, Ugyen Lhatshok, +3 authors Richa Awasthy • Published in International Conference on... 18 September 2025 • Computer Science, Education

TLDR This study presents a systematic review of peer-reviewed studies published between 2010 and 2024, focusing specifically on two critical dimensions of SA education: the teaching strategies employed and the challenges faced by students and educators.[Expand](#)

[View on IEEE](#) [doi.org](#) [Save to Library](#) [Create Alert](#) [Cite](#)

Figure 16: <https://www.semanticscholar.org/>

Personas (4/5)

Related Papers

Fostering self-regulated learning in higher education: Making self-regulation visible

J. M. Russell Chi Baik A. Ryan E. Molloy

Education, Psychology · Active Learning in Higher Education

TLDR

👍 88 Save Alert

Developing preservice teachers' teaching engagement efficacy: A classroom managerial implication

Prince Yeboah Asare Samuel Kobina Amo

Education · Cogent Education

TLDR

👍 15 Save Alert

Figure 17: <https://www.semanticscholar.org/>

Personas (5/5)

- ▶ While it works well, they do not understand my “persona”; cf.

```
$ query -q "software architecture teaching" \  
      -topCitedMax 10 -f BibTeX --withDOI \  
      --AllPDFsIfAvailableAsOpenAccess
```

```
$ query -q "software architecture teaching" \  
      --getRSSfeed --withRelatedPapers 3 \  
      --withPersonalization --maxPerDay 10
```

- ▶ Thus, you can also quickly lose users without understanding their needs and what they consider the TAM model's usefulness and ease of use to be

Ethics of Usability (1/3)

- ▶ Yet, what is being maximized is not always entirely clear....

The Dark Art of Dopamine Engineering

Jukka Ruohonen

University of Southern Denmark

Sønderborg, Denmark

juk@mmmi.sdu.dk

Qusai Ramadan

University of Southern Denmark

Sønderborg, Denmark

qura@mmmi.sdu.dk

Adam Alami

University of Southern Denmark

Sønderborg, Denmark

adal@mmmi.sdu.dk

Figure 18: Ruohonen et al. (2026; forthcoming; should be online soon)

Ethics of Usability (2/3)

- ▶ Though, it is becoming a little clearer

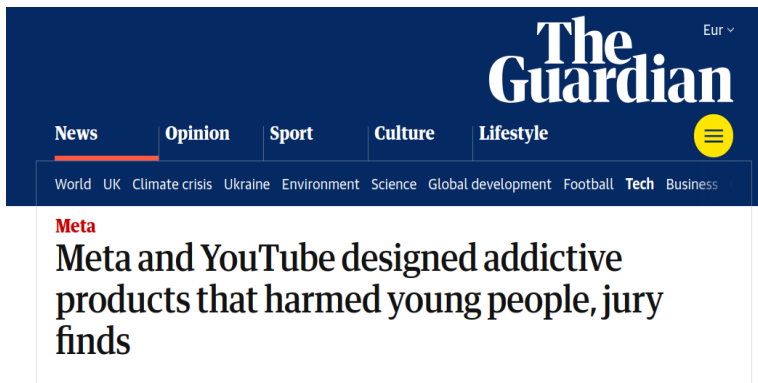


Figure 19: <https://www.theguardian.com/media/2026/mar/25/jury-verdict-us-first-social-media-addiction-trial-meta-youtube>

Ethics of Usability (3/3)

- ▶ Actually, it has long been clear

Deceptive Patterns



What are deceptive patterns?

Deceptive patterns (also known as “dark patterns”) are tricks used in websites and apps that make you do things that you didn't mean to, like buying or signing up for something. For example:

- ⚡ Trick wording
- ⚡ Sneaking
- ⚡ Obstruction

Figure 20: <https://www.deceptive.design/>

Exercise (20 minutes)

- ▶ Skim through the following award-winning paper:
<https://dl.acm.org/doi/pdf/10.1145/3359183>
- ▶ Questions:
 1. What do they say about people's cognitive biases?
 2. What can you say about the architecture in Fig. 1?
 3. Would you be able to re-implement their data collection process described in Subsection 4.2?
 4. What is your own experience with the dark patterns enumerated in Table 2?

Question

What Trade-Offs Usability Has?

Usability and Power Consumption

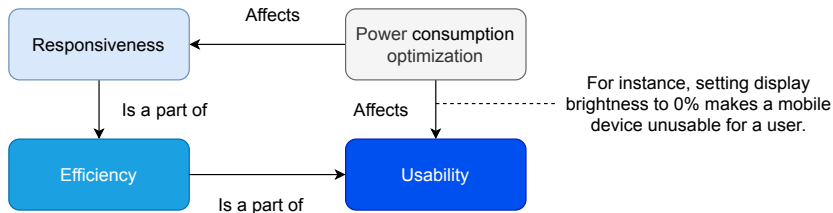


Figure 21: Optimizing Power Consumption viz Usability

Sustainability (1/2)

Dismantling public values, one data center at the time

In this blog post Julia Velkova discusses the problematics related to data centres from the Nordic perspective: Are the Nordic States too reliant on tech giants to in the critical communication infrastructures of the Welfare state?



AWS data center in Katrineholm, Sweden, October 2023. Besides lacking signage, this data center is the largest electricity consumer in the town, taking 1/4 of the capacity of the grid. (Image: Julia Velkova)

19.2.2024

Julia Velkova

NEWS

ENERGY

Share this page



Newsletter

Subscribe to
newsletter

Figure 22: <https://www.helsinki.fi/en/researchgroups/reimagining-public-values-in-algorithmic-futures/whats-new/dismantling-public-values-one-data-center-at-the-time>

Sustainability (2/2)

- ▶ Thus, sustainability and energy efficiency are hot topics also in technology politics of many countries
- ▶ When optimizing power consumption of a larger system, such as a data center, architectural choices are what matter rather than the optimization of software running in the system
- ▶ Therefore, in what follows, the focus is on a narrow framing toward **power consumption**, which is important for many consumer devices in particular but also elsewhere
 - In addition, the focus is on patterns and tactics rather than fine-grained optimizations at the level of detailed design

An x86 Example (1/9)

- ▶ The concept of **throttling** from the previous lecture is somewhat misleading because nowadays, on x86_64 at least, it is used for emergency situations involving **overheating**
 - In essence, it bluntly reduces the clock tick of CPUs
- ▶ Instead, both AMD and Intel nowadays seek to limit the power consumption of CPUs via so-called **P-states** and **C-states**
 - Others (such as ARM-vendors) do it too but the terminology might be slightly different at the hardware level

An x86 Example (2/9)

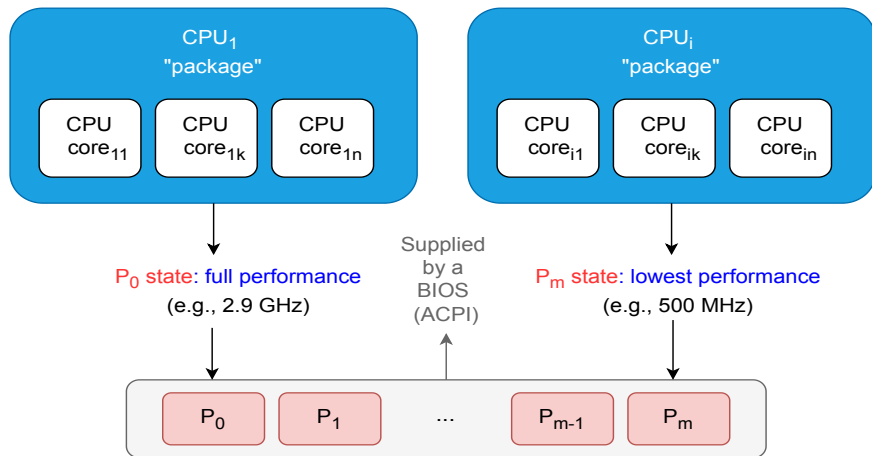


Figure 23: An Illustration of “Per-CPU-Package P-states”

An x86 Example (3/9)

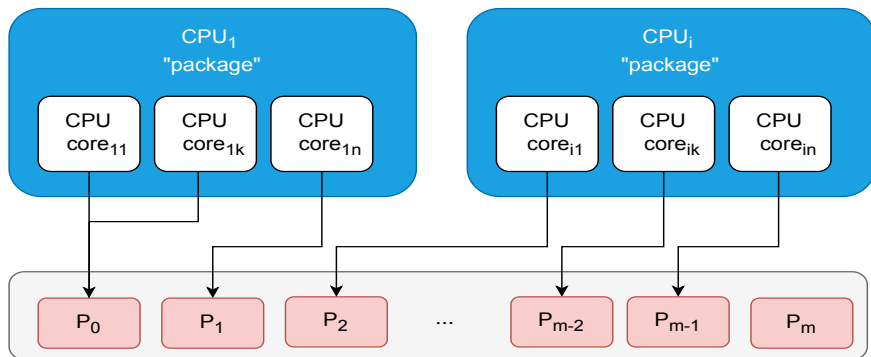


Figure 24: An Illustration of "Per-CPU-Core P-states"

An x86 Example (4/9)

PowerTOP 2.15		Overview	Idle stats	Frequency stats	Device stats	Tunables
Idle	Package		Core		CPU 0	CPU 4
				Average	2.4 GHz	2.6 GHz
		Idle		Idle		
			Core		CPU 1	CPU 5
				Average	2.4 GHz	2.2 GHz
		Idle		Idle		
			Core		CPU 2	CPU 6
				Average	2.7 GHz	2.3 GHz
		Idle		Idle		
			Core		CPU 3	CPU 7
				Average	2.4 GHz	2.2 GHz
		Idle		Idle		

Figure 25: <https://github.com/fenrus75/powertop>

An x86 Example (5/9)

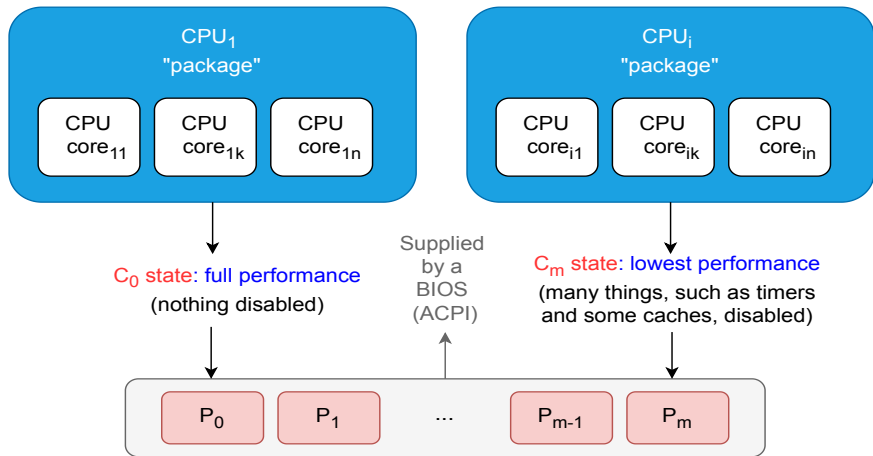


Figure 26: An Illustration of "Per-CPU-Package C-states"

An x86 Example (6/9)

PowerTOP 2.15 Overview Idle stats Frequency stats Device stats Tunables WakeUp									
Pkg(HW)		Core(HW)		CPU(OS) 0		CPU(OS) 4			
				C0 active	14.0%	16.5%			
				POLL	0.0%	0.0 ms	0.0%	0.0 ms	
				C1	0.3%	0.1 ms	0.0%	0.0 ms	
C2 (pc2)	0.0%	C3 (cc3)	1.2%	C3	2.2%	0.1 ms	0.8%	0.1 ms	
C3 (pc3)	0.0%		25.3%	C6	25.7%	0.4 ms	16.4%	0.5 ms	
C6 (pc6)	0.0%		39.7%	C7s	0.0%	0.0 ms	0.0%	0.0 ms	
C7 (pc7)	0.0%			C8	17.7%	0.7 ms	18.0%	0.8 ms	
C8 (pc8)	0.0%			C9	6.4%	1.4 ms	1.4%	1.5 ms	
C9 (pc9)	0.0%								
C10 (pc10)	0.0%			C10	31.2%	0.8 ms	44.9%	1.5 ms	
				C1E	1.5%	0.1 ms	0.5%	0.1 ms	
		Core(HW)		CPU(OS) 1		CPU(OS) 5			
				C0 active	40.3%	14.2%			
				POLL	0.0%	0.0 ms	0.0%	0.0 ms	
				C1	0.3%	0.2 ms	0.1%	0.1 ms	

Figure 27: <https://github.com/fenrus75/powertop>

An x86 Example (7/9)

PowerTOP 2.15 Overview Idle stats Frequency stats Device stats Tunables

System baseline power is estimated at 18.6 W

Power est.	Usage	Device name
8.82 W	4.1%	Display backlight
6.15 W	100.9%	CPU core
3.09 W	100.9%	CPU misc
538 mW	100.9%	DRAM
0 mW	1.0 pkts/s	nic:vnet11
0 mW	0.0 pkts/s	nic:virbr0
0 mW	100.0%	Radio device: iwlwifi
0 mW	0.0%	USB device: Integrated Camera (Azurewave)
0 mW	0.0%	USB device: usb-device-06cb-00bd
0 mW	0.0%	USB device: xHCI Host Controller
0 mW	0.0%	USB device: xHCI Host Controller
0 mW	0.0%	USB device: EMV Smartcard Reader (Generic)
0 mW	0.0%	USB device: xHCI Host Controller
0 mW	0.0%	USB device: xHCI Host Controller
0 mW	0.0 pkts/s	nic:docker0
0 mW	0.0 pkts/s	Network interface: wlp0s20f3 (iwlwifi)

Figure 28: <https://github.com/fenrus75/powertop>

An x86 Example (8/9)

- ▶ Thus, is **hardware that wakes up CPUs** and prevents them from entering into deeper C-states, and not only due to user input with the laptop, but also due to seldom used hardware (cf. USB) being unnecessarily 100% powered up
- ▶ Therefore, kernels try to shutoff as much hardware as possible
- ▶ You can also tune a lot further yourself
 - Say, `/tmp` and `/var/tmp` on a Linux/Unix laptop could be mounted as memory-based `tmpfs`s instead of using a disk

An x86 Example (9/9)



hdparm(8) - Linux man page

Name

hdparm - get/set SATA/IDE device parameters

Synopsis

hdparm [flags] [device] ..

Figure 29: <https://linux.die.net/man/8/hdparm>

- ▶ While much power can be saved by shutting things off, the earlier screenshots indicates that display in a laptop is actually the most power-hungry hardware component
 - Hence, **also better hardware is required**, and if batteries would see revolutionary innovations, the optimization trade-offs involved might not even perhaps be worth it?

Question

Which Tactic from the Past Lectures Might Be Used to Analytically Characterize P-states and C-states?

Another Question

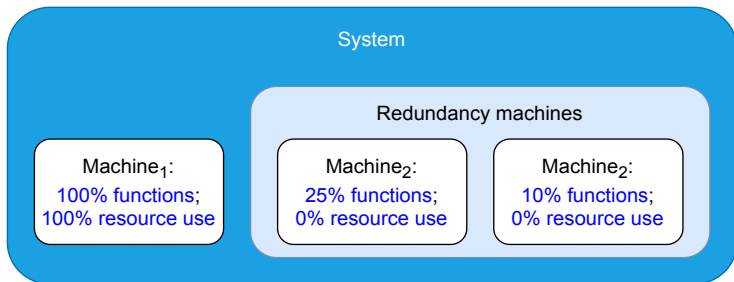
With Which Tactic(s) from the Past Lectures Does
Power Consumption Optimization Contradict?

Power Consumption Analytically (1/2)

- ▶ The previous example further highlights that power consumption can be analytically seen to contain two parts: “**computing power**” and “**idle power**”
 - Even when shutting down hardware, the idle power has a lower bound; at the extreme, it is what is minimally needed to run a device or a larger system
- ▶ When considering larger systems with $i = 1, 2, \dots, n$ machines, Ahmad & Vijaykumar (2010) further consider:

$$\begin{aligned} \text{Total power}_i &= \text{computing power}_i + \text{idle power}_i \\ &\quad + \text{cooling power}_i + \text{power loss}_i \end{aligned}$$

Power Consumption Analytically (2/2)



In addition, note that the power required to cool the system varies due to varying levels of hardware disabled and resources used.

Figure 30: Optimization Can Be Challenging Already with Small Systems!

Graceful Shutdown

- ▶ When a power runs out on a battery-powered device, an operating system should gracefully shutdown the device
- ▶ It is useful to provide **warnings** to users beforehand
- ▶ In addition, some operating systems and their configurations (such as those powering mobile phones) may gracefully shutdown functionalities when a battery is low
 - Hence, “graceful shutdown” $\neq$ “graceful degradation”
 - Also: “graceful degradation” $\neq$ “forceful degradation”

Question

What Example from the Past Lectures Could Be Used for the “Forceful Degradation” Notion?

Graceful Degradation (1/2)

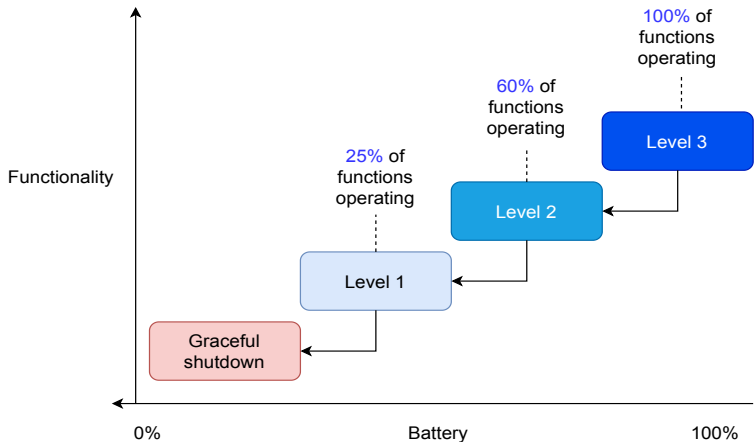


Figure 31: Graceful Degradation Revisited

Graceful Degradation (2/2)

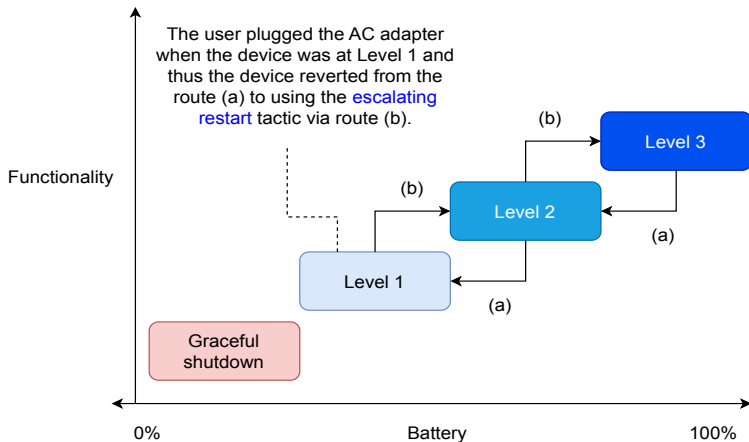


Figure 32: Mixing Two Tactics

Preloading (1/2)

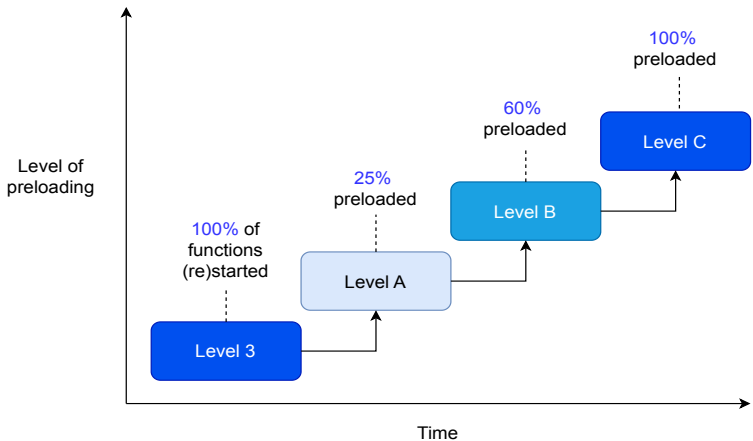


Figure 33: Using the Escalating Restart Tactic for Preloading

Preloading (2/2)

- ▶ Preloading can save power because **startup is typically resource-intensive** in many contexts
 - As it helps with **responsiveness** too, it can be seen as a way to deal with the earlier trade-off too (Hort et al. 2022)
- ▶ Analogously, it may sometimes be reasonable to never fully terminate lightweight processes even in case a user requests so
- ▶ What is being preloaded differs; on a mobile device it could be applications, while in cloud computing it might be data
- ▶ **Predictive analytics** can be used to determine what should be preloaded; e.g., on a mobile device, only a user's frequently used applications (based on historical data) could be preloaded

Question

In Which Context Preloading Was
Recommended Also in the Previous Lectures?

Offloading and Prefetching Are Common Tactics Too (1/2)

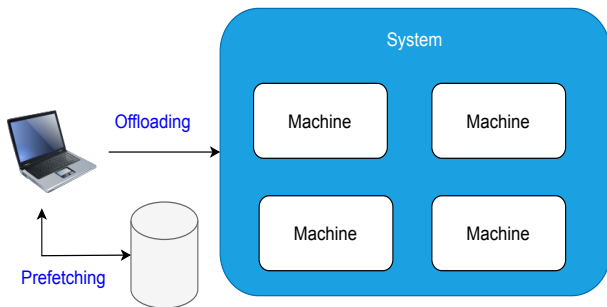


Figure 34: Two Concepts, Naïvely

Offloading and Prefetching Are Common Tactics Too (2/2)



BENEFIT ▾

CONTRIBUTE ▾

DISCOVER ▾

CARES

BLOG

Fourth Data Prefetching Championship: Part I

by Digvijay Singh on Apr 27, 2026 | Tags: Data Prefetcher, Memory Wall



Figure 35: <https://www.sigarch.org/fourth-data-prefetching-championship-part-i/>

Optimizations More Generally (1/2)

- ▶ From the previous exposition, we can deduce a general rule of thumb for developers seeking to optimize power consumption: try to *minimize software's actions that interfere with optimizations already done for hardware*; hence, e.g.,
 - Do not cause **unnecessary interrupts** that lead to hardware wake-ups (cf. notifications, polling, etc.)
 - Do not unnecessarily utilize a disk and a network
 - Use preloading, prefetching, and offloading when reasonable
 - Try to **avoid garbage collector runs** (Hort et al. 2022)
 - Note that **memory is hardware** too, and, therefore, many classical bug types (such as memory leaks) are often also power optimization bugs (ibid.)

Optimizations More Generally (2/2)

- ▶ Research shows that **programming languages** affect power consumption; software written in C consumes much less power than software written in Python, and so forth
- ▶ Thus, there is a further, more general trade-off
 - But is it worth it? Do the other trade-offs, such as those related to security, outweigh this particular trade-off?
- ▶ Trade-offs are what must be assessed also during **reviewing** software architectures and making decisions about them

New QAs (1/2)

- ▶ Sometimes software architects may need to **introduce a new QA** for which no existing body of knowledge exists
- ▶ Due to technological and other changes, it could even be argued that new QAs are even quite often needed
 - How to integrate AI/LLMs/Agents/whatnot into an existing enterprise software architecture? How to modernize a legacy architecture without redoing it from scratch? And so forth.
- ▶ Unsurprisingly, Bass et al. (2021) recommend starting with different **scenarios** (cf. sources, events, environments, systems and deployments, responses, and response measures)

New QAs (2/2)

- ▶ After having some scenarios for new architecturally significant requirements from stakeholders or elsewhere (such as law), Bass et al. (2021) suggest to:
 1. *Are existing scenarios, tactics, and patterns transferable, possibly with some modifications, to satisfy the new QA?*
 2. *What does academic literature say about the new QA? How about industry “gray literature”, standards, and other sources?*
 3. *What stakeholders, experts, and others say about the new QA?*
- ▶ Note that if you can “solve” the new QA before others or better than others, it can also be a business advantage
 - On the other hand: sometimes you might consider a reference architecture if the QA affects many companies

Evaluating Architectures and Designs (1/6)

- ▶ Architecture evaluation is a “*process of determining the degree to which an architecture is fit for the purpose for which it is intended*” (Bass et al 2021)
- ▶ Architecture evaluation is also a **risk reduction action**
 - Whether it is about cyber security risks or business risks, risks must be identified, and after they have been identified, addressing the risks is a **cost/benefit** action (ibid.)
 - Note that the noun “cost” should be understood loosely; thus, recall the notion of **acceptable risks**, which also implies that there are sometimes **unacceptable risks**

Evaluating Architectures and Designs (2/6)

- ▶ Bass et al. (2021) recommend:
 1. A group of reviewers “**individually** ensure that they **understand** the current state of the architecture”
 2. The reviewer group determines beforehand **evaluation criteria** (incl. metrics if possible) to guide the review
 3. “For each **scenario**, each reviewer should determine whether the scenario is satisfied”
 4. The reviewers seek to capture potential **problems** and identify **risks** exposed during the prior scenario-based step
- ▶ Although a plural form is used above, a single architect may also conduct reviews, and further note that outsiders may be involved in a peer review by multiple professionals

Evaluating Architectures and Designs (3/6)

- ▶ While a sensible recommendation, scenarios are only one available option; custom evaluation criteria are often used too
- ▶ Falessi et al. (2010) list numerous factors – among these are:
 1. Is a design **documented** and was it **reviewed**?
 2. What **business factors** influenced a design?
 3. What is a design's **return on investment**?
 4. Was a design's **security implications** addressed?
 5. How **useful** a design is to other software engineers?
 6. How much implementation **freedom** does a design constrain?
 7. How the development **team** behind a design was composed?

Evaluating Architectures and Designs (4/6)

- ▶ The previous points apply also to scientific research; thus, it should be understood that *software architecture is a research domain in software engineering as a scientific discipline*
- ▶ For instance, ACM defines four criteria for evaluating artifacts (documentation, consistency, completeness, and execution)

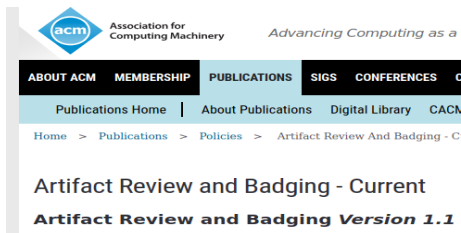


Figure 36: <https://www.acm.org/publications/policies/artifact-review-and-badging-current>

Evaluating Architectures and Designs (5/6)

- ▶ Rams (2023), a famous industrial designer, formulated ten principles for good industrial design, some of which resonate well also with software design (and some perhaps not):
 1. **Innovative**: a design always aligns with technology, and can thus never be an end in itself
 2. **Usefulness**: a good design is useful, but not only in terms of functionality, but also in terms of aesthetic quality and psychological aspects
 3. A good design is **aesthetic**
 4. **Understandability**: a good design clarifies a product's goals, structure, and functionality
 5. **Honesty**: a good design does not manipulate users
 6. A good design is **thorough** to the last detail
 7. A good design is **environmentally friendly**
 8. A good design involves as **little design as possible**

Evaluating Architectures and Designs (6/6)

- ▶ The output from an evaluation should be documented
- ▶ Bass et al. (2021) recommend to focus on:
 1. A **concise overall presentation** of the architecture
 2. Articulation of **business or other goals**
 3. **Prioritization of quality attributes** based on scenarios
 4. A set of **risks**, including acceptable risks and non-risks
 5. **Mapping** of architectural decisions to quality attributes
 6. Articulation of **trade-offs** and decisions thereto
- ▶ For reusability and rigor, it may be sometimes a good idea to document in **tabular form** (e.g., having architectural decisions as rows and risks, trade-offs, and impacts as columns)

Exercise (20 minutes)

- ▶ Skim through the following award-winning, modern classic in computer science from 1983:
<https://dl.acm.org/doi/epdf/10.1145/800217.806614>
- ▶ Questions:
 1. What design principles can you identify?
 2. Which points you like and can agree with?
 3. Anything you disagree with? If so, why?

Questions?

Additional References (1/2)

1. Hertzum, M. & Hornæk (2023): Frustration: Still a Common User Experience. ACM Transactions on Computer-Human Interaction 30(3), <https://dl.acm.org/doi/pdf/10.1145/3582432>
2. Davis, F. D. (1989): Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology Available to Purchase. MIS Quarterly 13(3), <https://doi.org/10.2307/249008>
3. Silva, S. (2023): Quality Metrics in Software Architecture. ICSA, <https://doi.org/10.1109/ICSA56044.2023.00014>
4. Ahmad, F. & Vijaykumar, T. N. (2010): Joint Optimization of Idle and Cooling Power in Data Centers While Maintaining Response Time. ACM SIGPLAN Notices, <https://dl.acm.org/doi/pdf/10.1145/1735971.1736048>

Additional References (2/2)

1. Hort, M. & Kechagia, M. & Sarro, F. & Harman, M. (2022): A Survey of Performance Optimization for Mobile Applications. IEEE Transactions on Software Engineering, <https://doi.org/10.1109/TSE.2021.3071193>
2. Falessi, D. & Ali Babar, M. & Cantone, G. & Kruchten, P. (201): Empirically Assessing Software Architecture Research: Challenges and Lessons Learned, Empirical Software Engineering 15(3), pp. 250–276, <https://arxiv.org/pdf/1701.06000>
3. Rams, D. (2023): Dieter Rams and the 10 Principles for a Good Design. Domus, <https://www.domusweb.it/en/from-the-archive/2023/03/14/dieter-rams-and-the-10-principles-for-a-good-design.html>